

Visualization using Paraview

Paraview is an open-source, multi-platform data analysis and visualization tool with wide support on most large-scale computers. We use it extensively for simulation setup, monitoring, and post-processing.

Two operational modes are possible:

1. **Standalone:** Use to visualize data available on your local ("client") machine (e.g., workstation or laptop)
2. **Client-server:** Use to visualize data hosted by a remote server (e.g., CRC cluster, DOE/DOD supercomputer, etc.)

Paraview's strength is visualizing remote data. To do this, it operates in client-server mode, wherein the user initiates both a remote **server** process (e.g., using several MPI ranks on the data-bearing machine) and a local **client** process on their workstation/laptop. We will discuss this setup below.

If you are trying to visualize on a Navy DSRC machine, follow the instructions on [this page](#).

Installing the Paraview client

A local installation of Paraview is necessary for use in both standalone and client-server modes. Binary executables for Linux, Windows, and macOS may be downloaded from the [Paraview website](#).

A note on version numbers: The client and server version numbers must match exactly, i.e. including the minor version number *Z* in *vX.Y.Z*. To determine the versions available on the server you are using, use `module avail` on the remote machine and look for "Paraview."

On CRC machines, the recommended version is `paraview/5.9.1` (as of 12/6/24). This means that you should download `v5.9.1` from the [Paraview website](#). (Linux users should download the Python3 version, if it is an option.) Other computing centers will likely have different default Paraview versions.

Unpack the tar file to a known location and verify that the executable `paraview` is available in the `bin` folder.

Paraview may also be compiled from source. However, the binaries should be sufficient for most use cases.

Visualizing *PyFlowCL* data

This section describes how to load *PyFlowCL* data using the Paraview client. For data on remote machines, you will want to start a client-server connection first, which is described in the next section.

In the local Paraview client window, select `File -> Open`. Navigate to your data output directory. You should see two sets of files:

1. `.h5` files: binary grid and data fields
2. `.xmff` files: XML-format metadata

Paraview should collapse similarly-named files. Select the collapsed `.xmf` and click **Open**. A dialog box will appear asking which reader to use; select the one named **XDMF Reader**. Click **Apply** in the left-hand pane. Your data is now loaded.

Using Paraview in client-server mode

We will use the CRC's Epyc machine as an example. In general, this tutorial may be adapted to any remote machine.

Serial visualization

This is the basic setup for **serial** client-server visualization. This works for small data files (<1 GB,) but, for larger files, parallel remote server can be necessary. These instructions cover the basic usage of port forwarding, which is then extended to parallel client-server operation in the next section.

Follow these instructions to run a serial visualization server on the remote machine.

- Log on to `epycfe.crc.nd.edu` with the local (workstation/laptop) port 11111 forwarded. For off-campus visualization, the port forwarding may be daisy-chained as necessary.

```
ssh -L 11111:localhost:11111 <NetID>@epycfe.crc.nd.edu
```

- Upon connecting, load the Paraview module and start a server:

```
module load paraview/5.9.1
pvserver
```

- On the local machine, start the client using the following command, with the path to your local Paraview installation set appropriately.

```
/path/to/ParaView/bin/paraview
```

In certain cases, it might be necessary to preload a C++ library.

```
LD_PRELOAD=/usr/lib64/libstdc++.so.6 /path/to/ParaView/bin/paraview
```

- Click File > Connect. In the "connect to server" dialog (click "Add Server"), specify the server as `localhost` and the port as `11111`.
- Click connect. In the remote terminal (`epycfe.crc.nd.edu`, for example), `pvserver` should say "Client connected."
- In the "open file" dialog, navigate to a the `.xmf` files, open them, and visualize away.

Parallel visualization

This is for visualizing large simulation results and only works for *parallel* file systems, such as scratch365 (**not** AFS).

- Log on to epycfe.crc.nd.edu **without** port forwarding. We will forward a port later.
- Request an interactive job allocation. For example, to visualize on up to 64 AMD Epyc cores,

```
qrsh -q hpc -pe mpi-64 64
```

You will be logged in to your assigned compute node. From here, load the Paraview module and initialize [pvserver](#) using an appropriate number of MPI tasks. **NOTE: More cores is not always better!** The communications overhead of larger MPI jobs can slow Paraview down. The optimal number of cores will depend on the size of the data being visualized. For example, to visualize on 2 cores (out of 64, if the above command was used),

```
module load paraview/5.9.1  
mpirun -np 2 pvserver
```

- Take note of the specific remote server on which you started [pvserver](#). For example, if your remote terminal says

```
Accepting connection(s): d24cepyc127.crc.nd.edu:11111
```

then the remote server is [d24cepyc127](#).

- Open a **new terminal window** on your local machine. We will now log in to the *head node* of the remote machine--external login to compute nodes is typically disallowed---while specifying the port forwarding that we need to reach the compute node. For example, if we ran [pvserver](#) on [d24cepyc127](#),

```
ssh -L 11111:d24cepyc127:11111 <NetID>@epycfe.crc.nd.edu
```

- Now, open Paraview on your client machine and connect to the server as in the serial client-server instructions above. If successful, you should get a **Client Connected** message in the terminal in which you ran [pvserver](#).
- Be sure to end your interactive job via [qdel](#) to free up resources for others.

Note: The second terminal window (with port forwarding) must be closed before port 11111 can be forwarded to a different machine. Alternatively, you may specify a different port via [pvserver](#) and adjust your forwarded port accordingly, e.g., on the server:

```
pvserver --server-port=11112
```

and on the client:

```
ssh -L 11111:localhost:11112 <NetID>@epycfe.crc.nd.edu
```